PROCESS-MEMORY

코드·데이터·힙·스택과 메모리 오류

연관 기출 · 138-2-6, 136-3-6, 136-1-7

한눈에 이해

실행할 명령, 계속 유지할 값, 동적으로 얻은 값, 함수 호출의 임시값은 수명과 관리 방법이 다릅니다.

정의와 핵심 원리

일반적인 프로세스의 주소 공간은 실행 코드, 정적 데이터, 동적 힙, 호출 스택 같은 논리 영역으로 나뉩니다. 실행 파일과 메모리 매핑의 구성은 운영체제·언어·실행 환경에 따라 달라집니다.

• 코드 영역에는 실행할 명령이 놓입니다. 읽기 전용 자료는 별도 영역에 둘 수도 있습니다. 데이터 영역에는 초기화된 전역·정적 변수와 BSS 같은 영역이 포함됩니다. 힙에는 동적으로 할당한 객체를 두고, 스택은 함수 호출마다 인자·지역 변수·복귀 정보를 프레임으로 보관합니다. 최적화·클로저·코루틴 등에 따라 지역 변수가 스택 바깥에 놓일 수 있습니다.

• C에서 malloc은 메모리를 할당하고 free는 반환합니다. 할당한 주소를 잃어 반환하지 못하면 메모리 누수가 생깁니다. 반환한 뒤 다시 접근하면 use-after-free, 두 번 반환하면 double-free입니다. Java나 Python에서도 컬렉션·캐시·리스너가 더 이상 필요 없는 객체를 계속 참조하면 가비지 컬렉터(GC)가 회수할 수 없습니다.

• 세그멘테이션 오류는 허용되지 않은 메모리에 접근할 때 발생할 수 있습니다. 널 포인터(NULL) 역참조, 경계를 벗어난 접근, 해제 후 접근 등이 원인 후보입니다. 누수는 사용량 추세·힙 덤프·참조 경로를, 보호 위반은 덤프·스택 추적·메모리 검사 결과를 살핍니다.

구조·비교·판단

구분	핵심 원리	적용·한계
코드·데이터	프로그램 명령과 정적 수명의 데이터	권한과 초기화 여부 구분
힙	동적으로 할당한 객체의 수명	누수·해제 후 사용에 주의
스택	호출 프레임의 수명	깊은 재귀, 함수가 끝난 뒤 지역 변수 참조에 주의

적용 예제

요청마다 쌓이는 캐시

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 요청 한 건마다 1 MiB 크기의 객체를 전역 리스트에 추가합니다. 요청은 1000번 들어옵니다.

1. 각 요청이 끝나도 전역 리스트는 객체를 계속 참조합니다.
2. 약 1000 MiB의 객체와 이를 관리하는 메모리가 남습니다.
3. 힙 덤프에서 메모리를 붙잡고 있는 객체와 참조 경로를 따라가 전역 리스트를 확인합니다.
4. 보관 기간과 항목 수의 한도를 정하고 기한이 지난 객체의 참조를 제거합니다.

결과·해석: GC 호출만 반복하는 대신 도달가능한 불필요 객체의 보유 원인을 수정합니다.

전제·주의: 주소 공간 그림은 개념 모형입니다. 힙·스택이 자라는 방향은 플랫폼에 따라 달라질 수 있습니다.

답안 골격과 혼동 방지

네 영역의 역할과 수명 → 호출과 할당의 흐름 → 누수와 보호 위반 구분 → 언어별 진단 → 재발 방지

- 가비지 컬렉터가 있어도 불필요한 객체에 대한 참조가 남으면 메모리가 누수될 수 있습니다.
- SIGSEGV가 발생하면 물리 메모리 부족을 단정하기 전에 접근한 주소와 권한을 확인합니다.

내용을 가리고 회상

1. 전역 리스트가 참조하는 힙 객체를 가비지 컬렉터가 회수하지 못하는 이유는 무엇입니까?
2. 스택의 지역 변수에 대한 참조를 함수 밖으로 반환하면 왜 위험합니까?
3. 메모리 누수와 해제 후 사용은 어떻게 다릅니까?

적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

내용 검토 상태: 부분 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- malloc(3) - Linux man-pages project; 상시 문서; 확인 2026-11-05.
- OSTEP: Segmentation - Remzi H. Arpaci-Dusseau; Andrea C. Arpaci-Dusseau / UW-Madison; 공개판 1.10; 확인 2026-11-05.